AI Dice Battle — CustomTkinter GUI Game
Detailed Project Documentation

Submitted By – Nikhil
Reg. no. – 25BME10013

Course – Fundamentals in AI and ML
Course Code – CSA2001

INTRODUCTION

AI Dice Battle is a turn-based desktop game developed in Python using the CustomTkinter library for its graphical interface.
The application allows a human player to compete against an AI opponent through strategic dice rolls that determine damage.
This project combines probability, game logic, artificial intelligence behavior, and GUI development into a single cohesive program.
It demonstrates how structured programming and user interface design can work together to produce an engaging and interactive software experience.

MOTIVATION OF THE PROJECT

The primary motivation for this project is to demonstrate how simple game rules can be enhanced using a graphical interface and AI-driven decision making.
Many beginner projects remain limited to console outputs and lack interactive design.
This project aims to bridge that gap by showing how Python can be used to create visually appealing applications while still maintaining clean and understandable logic.
It encourages learning through creativity, interactivity, and practical implementation of programming concepts.

PROBLEM STATEMENT

Most introductory Python games are text-based and do not provide an engaging user experience.
They lack graphical representation, real-time feedback, and intelligent opponent behavior.
This limits both the learning potential and the entertainment value of such programs.
The challenge addressed by this project is to transform a simple dice-based rule system into

a GUI-driven application that includes AI logic, user interaction, and clear visual feedback within a structured design.

OBJECTIVE OF THE PROJECT

The objective of this project is to design and develop a turn-based dice game with a modern graphical interface.
It aims to separate core game logic from GUI components while implementing basic artificial intelligence for decision making.
Another objective is to provide an intuitive user experience through buttons, logs, and real-time updates.
The project also intends to demonstrate modular coding practices and object-oriented design in Python applications.

EXISTING METHODS

Traditional dice games are typically played physically or implemented as simple console-based programs.
These methods rely heavily on manual interaction or text input and lack visual appeal.
Most implementations do not include intelligent behavior for automated opponents.
As a result, the user experience remains basic and less immersive.
Existing methods serve as a foundation but do not fully utilize modern GUI tools or programming techniques available in Python.

PROS AND CONS OF THE STATED METHODS

Console-based and traditional dice games are simple to understand and easy to implement with minimal system requirements.
They are ideal for learning basic programming concepts and game rules.
However, they lack graphical feedback, interactive design, and intelligent opponent strategies.
This makes them less engaging for users.
The absence of a visual interface reduces the overall experience and limits the practical exposure to GUI development techniques.

HARDWARE AND SOFTWARE REQUIREMENTS

The hardware requirements for this project are minimal, requiring only a basic computer system with at least 4GB RAM.
The software requirements include Python version 3.8 or higher and the CustomTkinter library for GUI development.
The project is platform-independent and can run on Windows, Linux, or macOS systems.
No additional external tools are required, making the project easy to set up and execute.

METHODOLOGY AND GOAL

The methodology followed involves first designing the core game logic functions such as dice rolling, scoring, and AI decisions.
After ensuring the correctness of these functions, they are integrated into a GUI class built with CustomTkinter.
The goal is to maintain separation between logic and interface while ensuring smooth user interaction.
This structured approach improves readability, scalability, and maintainability of the code.

FUNCTIONAL MODULES DESIGN AND ANALYSIS

The system is divided into functional modules including dice rolling, score calculation, AI decision making, GUI display, and game state management.
Each module performs a specific task and interacts with others in a controlled manner.
This modular design helps in debugging, testing, and future enhancement.
It also ensures that the program remains organized and easy to understand for developers.

ALGORITHM DEVELOPMENT

The algorithm begins by initializing player and AI health points.
The player selects the number of dice to roll, followed by score calculation based on special rules.
The AI then decides its dice count and rolls accordingly.
Scores are compared, and damage is applied to the losing side.
This loop continues until one participant's health reaches zero, determining the winner.

SOFTWARE ARCHITECTURAL DIAGRAM

The software architecture follows a modular and layered approach where the GUI interacts with independent logic functions.
The GUI class handles user interaction, display updates, and state tracking.
Logic functions remain separate to perform calculations and decisions.
This architecture ensures clean separation of concerns, making the application easier to maintain and extend with additional features in the future.

CODING

The project is implemented using Python and follows object-oriented programming principles.
Functions are used for core logic, while a class structure manages the GUI and state variables.
The CustomTkinter library provides modern widgets and styling options for the interface.
The random module simulates dice behavior.

Clear naming conventions and structured layout improve code readability and maintainability.

OUTPUT

The output of the project is a graphical window displaying player health, AI health, round number, and a live battle log.
Interactive buttons allow the player to choose dice rolls.
The interface updates dynamically after each round, providing real-time feedback.
When the game ends, the system displays the winner and disables further interaction.

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

Key implementations include GUI integration with logic functions, AI decision-making strategy, special scoring rules, and real-time state updates.
The system ensures smooth communication between interface and logic.
Game-over handling and button disabling are also implemented to maintain proper flow.
These implementations form the core functionality of the application.

SIGNIFICANT PROJECT OUTCOMES

The project successfully demonstrates how Python can be used to create interactive desktop games with GUI support.
It highlights the effectiveness of modular design and simple AI logic in enhancing user engagement.
The outcome is a functional, visually appealing application that combines learning and entertainment through structured programming.

TESTING AND REFINEMENT

The system was tested for correct dice scoring, AI behavior consistency, GUI responsiveness, and edge cases such as ties and health limits.
Multiple test runs ensured stability and accuracy.
Refinements were made to improve layout, readability, and user interaction.
This process ensured a reliable and user-friendly final application.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

The concepts used in this project can be applied to educational tools, beginner game development, and GUI-based desktop applications.
It serves as a model for integrating logic and interface design.
The approach can be extended to simulations, training software, and interactive learning platforms where user engagement is important.

CONTRIBUTION/ FINDINGS OF THE PROJECT

The project shows that simple rule-based systems can become engaging when combined with visual design and AI behavior.
It reinforces the importance of separating logic from interface.
The findings highlight how GUI frameworks can significantly enhance user experience in Python applications.

LIMITATION/CONSTRAINTS OF THE SYSTEM

The system has limited graphics and uses basic AI decision-making logic.
There is no persistent score tracking or multiplayer capability.
The design focuses on simplicity rather than advanced features.
These limitations provide opportunities for future enhancements and improvements.

CONCLUSIONS

The project successfully integrates GUI development and game logic into a single Python application.
It demonstrates practical use of CustomTkinter and modular coding practices.
The result is an engaging and educational game that showcases how structured programming can produce interactive software.

FUTURE ENHANCEMENTS

Future improvements may include adding dice images, sound effects, advanced AI strategies, restart options, and score history tracking.
Enhancing visual design and adding animations could further improve user engagement.
These enhancements would transform the project into a more complete and polished game application.

REFERENCES

Python Official Documentation.
CustomTkinter Library Documentation.
Basic Game Design Principles and GUI Programming Concepts from standard programming resources and tutorials.